

HW1_Problem2

January 28, 2026

1 Problem 2

```
[ ]: import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import random
from sklearn.model_selection import train_test_split, cross_val_score, \
    ↪StratifiedKFold, cross_val_predict
from sklearn.linear_model import SGDClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.metrics import confusion_matrix, recall_score, precision_score, \
    ↪accuracy_score
from sklearn.dummy import DummyClassifier
```

1.1 2.1

1.1.1 Load the data as arrays and split them into training and test sets with the next ratio: 85-15. Verify that all the classes have the adequate number of instances.

Load the MNIST data

1.2 Fetch & Load MNIST Dataset

```
[ ]: from sklearn.datasets import fetch_openml
mnist = fetch_openml('mnist_784', as_frame = False)
```

Inspect the data

```
[ ]: X,y = mnist.data, mnist.target
print(f'{"-" * 30}\ndata array:\n {X}, \n\nshape : {X.shape}')
print(f'{"-" * 30}\ntarget array:\n {y}, \n\nshape : {y.shape}\n{"-" * 30}')
```

```
-----
data array:
[[0 0 0 ... 0 0 0]
 [0 0 0 ... 0 0 0]
 [0 0 0 ... 0 0 0]
 ...
```

```
[0 0 0 ... 0 0 0]
[0 0 0 ... 0 0 0]
[0 0 0 ... 0 0 0]],
```

```
shape : (70000, 784)
```

```
target array:
```

```
['5' '0' '4' ... '4' '5' '6'],
```

```
shape : (70000,)
```

Split into training and test sets

We split the dataset into:

85% training data.

15% test data.

Stratify=y ensures that the proportion of each class in the target variable y is maintained in both sets.

random_state=42 for reproducibility.

Why? → In classification tasks, stratification is essential to avoid creating unbalanced splits, especially when some classes are underrepresented.

```
[ ]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.15,
↳stratify=y, random_state=42)
```

We retrieve:

Unique class labels.

Their respective counts in both y_train and y_test.

We store these counts in dictionaries for easy inspection.

This step ensures that each class has a sufficient number of instances for meaningful training and evaluation.

```
[ ]: unique_labels, train_counts = np.unique(y_train, return_counts=True) # Get the
↳unique labels and their counts in the train set
train_dict = dict(zip(unique_labels, train_counts)) # Map each label to its
↳count and store them as a dictionary

unique_labels, test_counts = np.unique(y_test, return_counts=True)
test_dict = dict(zip(unique_labels, test_counts))

print(f'{train_dict}\n{test_dict}')
```

```
{'0': 5868, '1': 6695, '2': 5942, '3': 6070, '4': 5800, '5': 5366, '6': 5845,
'7': 6199, '8': 5801, '9': 5914}
```

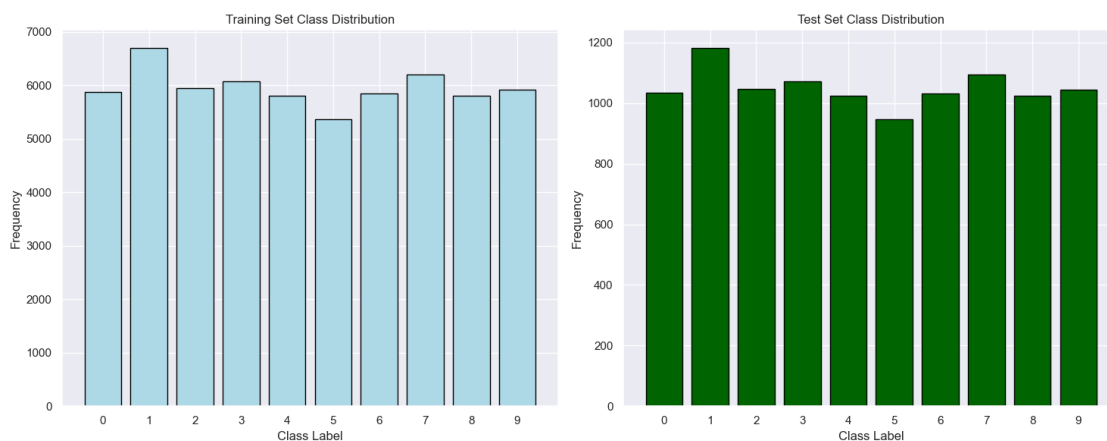
```
{'0': 1035, '1': 1182, '2': 1048, '3': 1071, '4': 1024, '5': 947, '6': 1031,
'7': 1094, '8': 1024, '9': 1044}
```

Create a visualization to confirm that all classes have the adequate number of instances.

```
[ ]: fig, axs = plt.subplots(1, 2, figsize=(15, 6)) # Set up the layout
axs[0].bar(train_dict.keys(), train_dict.values(), color='lightblue',
           edgecolor='black')
axs[0].set_xticks(range(10))
axs[0].set_title('Training Set Class Distribution')
axs[0].set_xlabel('Class Label')
axs[0].set_ylabel('Frequency')

axs[1].bar(test_dict.keys(), test_dict.values(), color='darkgreen',
           edgecolor='black')
axs[1].set_xticks(range(10))
axs[1].set_title('Test Set Class Distribution')
axs[1].set_xlabel('Class Label')
axs[1].set_ylabel('Frequency')

plt.tight_layout()
plt.show()
```



We observe that there is no class with very few or way too many instances, and our data is well balanced in both the training and the test set.

1.3 2.2

1.3.1 Depict the first 8 images of the created training and test sets using different subplots in a 2 by 4 frame, with their labels as titles.

Plot the first 8 images from the training set

```
[ ]: fig, axs = plt.subplots(2, 4, figsize=(10, 5))

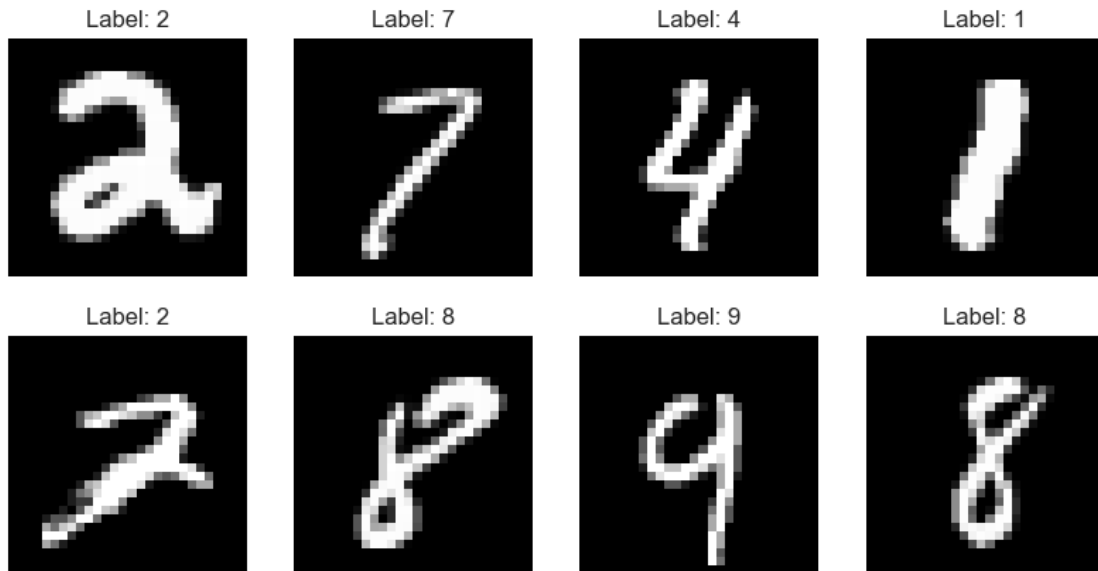
index = 0
for row in range(2):
    for col in range(4):
```

```

    axs[row, col].imshow(X_train[index].reshape(28, 28), cmap='gray')
    axs[row, col].set_title(f"Label: {y_train[index]}")
    axs[row, col].axis('off')
    index += 1

```

```
plt.show()
```



Plot the first 8 images from the test set

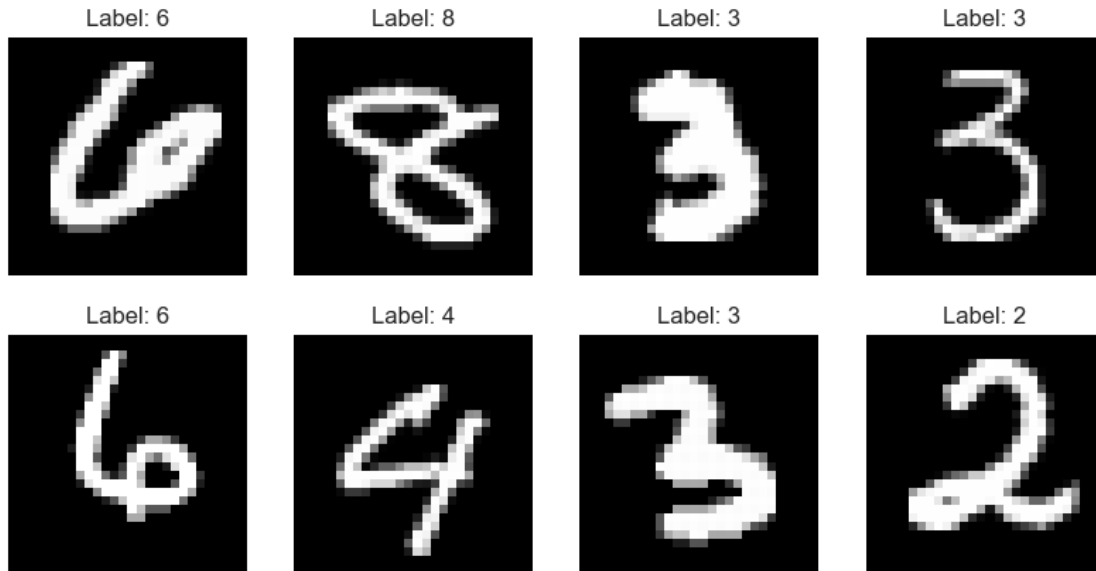
```

[ ]: fig, axs = plt.subplots(2, 4, figsize=(10, 5))

index = 0
for row in range(2):
    for col in range(4):
        axs[row, col].imshow(X_test[index].reshape(28, 28), cmap='gray')
        axs[row, col].set_title(f"Label: {y_test[index]}")
        axs[row, col].axis('off')
        index += 1

plt.show()

```



1.4 2.3

1.4.1 We need to handle a classification problem of distinguishing between two classes: even and odd numbers. First, create the training and test subsets for each class. Then, choose a binary classifier and a normalization technique of your choice, before wrapping them into a scikit-learn pipeline. Fit your pipeline to observe the created diagram.

Create binary labels for even and odd numbers

```
[ ]: y_train_binary = (y_train.astype(int) % 2 == 0) # If the remainder of the
    ↪ division with 2, is 0, then the value is an even
    # number (returns 1). If the
    ↪ remainder is 1, then it is an odd
    # number (returns 0).
y_test_binary = (y_test.astype(int) % 2 == 0)
```

Create a pipeline with StandardScaler and SGDClassifier. Then fit it on the training data

```
[ ]: pipeline = Pipeline([('scaler', StandardScaler()), ('sgd_clf',
    ↪ SGDClassifier(random_state=42))])

pipeline.fit(X_train, y_train_binary)
```

```
Pipeline(steps=[('scaler', StandardScaler()),
    ('sgd_clf', SGDClassifier(random_state=42))])
```

1.5 2.4

1.5.1 Use 3-fold cross validation and evaluate your classification pipeline by calculating the next metrics: accuracy, recall, and precision. Compare the predictive performance of your model against a dummy model that always guesses that an image belongs to the even category.

Evaluate accuracy, recall, and precision using 3-fold cross-validation

```
[ ]: accuracy = cross_val_score(pipeline, X_train, y_train_binary, cv=3,
    ↳scoring='accuracy')
recall = cross_val_score(pipeline, X_train, y_train_binary, cv=3,
    ↳scoring='recall')
precision = cross_val_score(pipeline, X_train, y_train_binary, cv=3,
    ↳scoring='precision')

print(f'Model Accuracy: {round(accuracy.mean()*100,2)}%')
print(f'Model Recall: {round(recall.mean()*100,2)}%')
print(f'Model Precision: {round(precision.mean()*100,2)}%')
```

Model Accuracy: 88.33%

Model Recall: 87.67%

Model Precision: 88.48%

Create a dummy classifier that always predicts “even”

```
[ ]: dummy_clf = DummyClassifier(strategy = 'constant', constant = True) # We want
    ↳the model to always predict even (1), therefore,
                                                    # we use
    ↳strategy='constant' and set the constant value to
                                                    # True = 1
    ↳= even
dummy_clf.fit(X_train, y_train_binary) # Fit the dummy classifier to the
    ↳training data
dummy_accuracy = cross_val_score(dummy_clf, X_train, y_train_binary, cv=3,
    ↳scoring='accuracy') # Calculate its accuracy.

print(f'Dummy Model Accuracy: {round(dummy_accuracy.mean()*100,2)}%')
```

Dummy Model Accuracy: 49.17%

The Dummy Classifier always predicts ‘even’. Given the dataset is split into $N(\text{even})$ even and $N(\text{odd}) = n - N(\text{even})$ odd values, it will predict:

$TP = N(\text{even})$, $FP = N(\text{odd})$, $TN = FN = 0$, with $N(\text{even}) + N(\text{odd}) = n$.

This means that:

$$\text{precision} = \frac{TP}{TP+FP} = \frac{N(\text{even})}{N(\text{even})+N(\text{odd})} = \frac{N(\text{even})}{n} = \text{accuracy}$$

and also

$$\text{recall} = \frac{TP}{TP+FN} = \frac{N(\text{even})}{N(\text{even})+0} = 1$$

Therefore, examining precision and recall for this dummy model adds no extra insight beyond what the accuracy tells us. ##### Comparing the performance of the two models The trained pipeline

exhibits much better predictive power compared to the dummy model. Its accuracy (88.33%) is far above random guessing. This tells us that the model is successfully learning to differentiate between even and odd digits in contrast to the dummy model, that lacks the ability to make informed predictions.

1.6 2.5

1.6.1 Calculate the confusion matrix for the training set, following the same 3-fold cross validation protocol. Record the kind and the amount of the predictions based on that.

Generate predictions using 3-fold cross-validation for the training set

```
[ ]: y_train_pred = cross_val_predict(pipeline, X_train, y_train_binary, cv = 3)
```

Calculate and print the confusion matrix

```
[ ]: cm = confusion_matrix(y_train_binary, y_train_pred)
      print(f'Confusion Matrix for the Training Set:\n{cm}')
```

Confusion Matrix for the Training Set:

```
[[26905  3339]
 [ 3607 25649]]
```

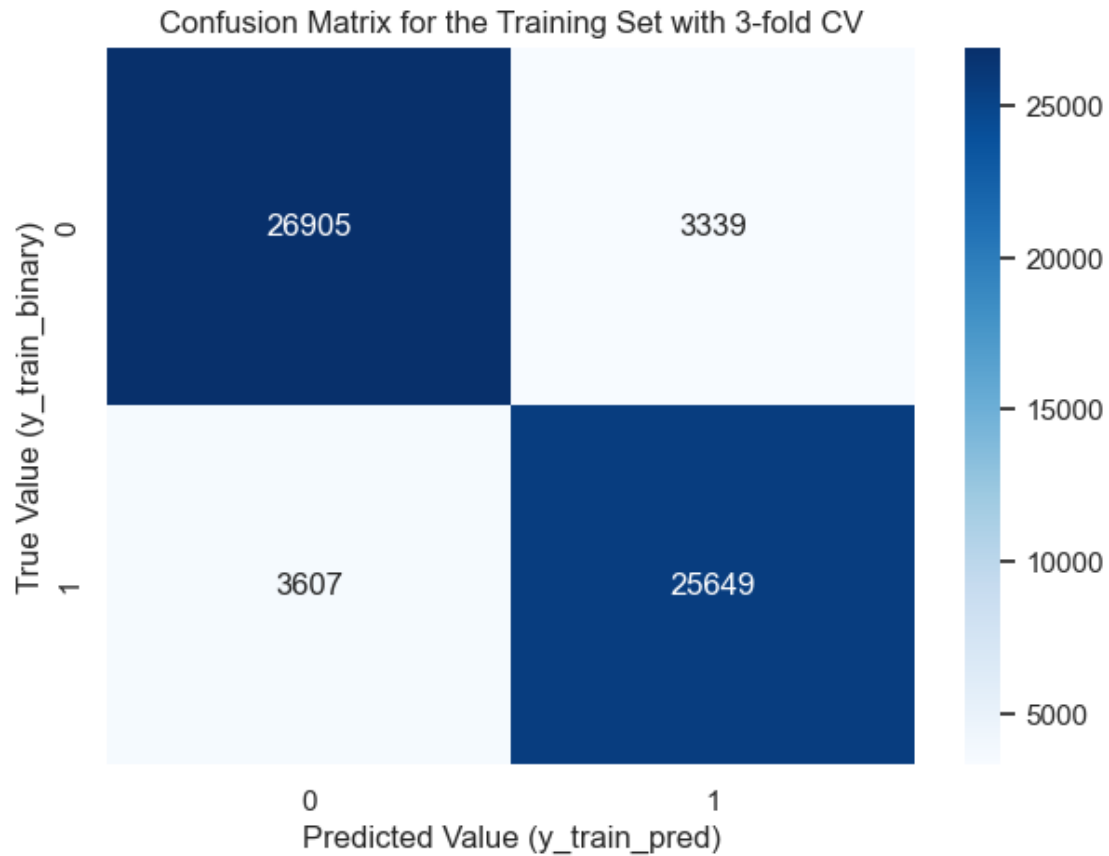
Extract and print the confusion matrix components

```
[ ]: tn, fp, fn, tp = cm.ravel()
      print(f"True Negatives (TN): {tn}, False Positives (FP): {fp}, False Negatives (FN): {fn}, True Positives (TP): {tp}")
```

True Negatives (TN): 26905, False Positives (FP): 3339, False Negatives (FN): 3607, True Positives (TP): 25649

Visualize the confusion matrix for better readability

```
[ ]: plt.figure(figsize=(7, 5))
      sns.heatmap(cm, annot=True, cmap='Blues', fmt='g')
      plt.title('Confusion Matrix for the Training Set with 3-fold CV')
      plt.xlabel('Predicted Value (y_train_pred)')
      plt.ylabel('True Value (y_train_binary)')
      plt.show()
```



1.7 2.6

1.7.1 Train the same pipeline over all the training set, and apply that on the test set for getting your predictions. Extract again the confusion matrix, and comment any great changes in the behavior of your model.

Train the classifier on the entire training set

```
[ ]: pipeline.fit(X_train, y_train_binary)

Pipeline(steps=[('scaler', StandardScaler()),
                 ('sgd_clf', SGDClassifier(random_state=42))])
```

Predict the labels on the test set

```
[ ]: y_test_pred = pipeline.predict(X_test)
```

Calculate and print the confusion matrix for the test set

```
[ ]: cm_test = confusion_matrix(y_test_binary, y_test_pred)

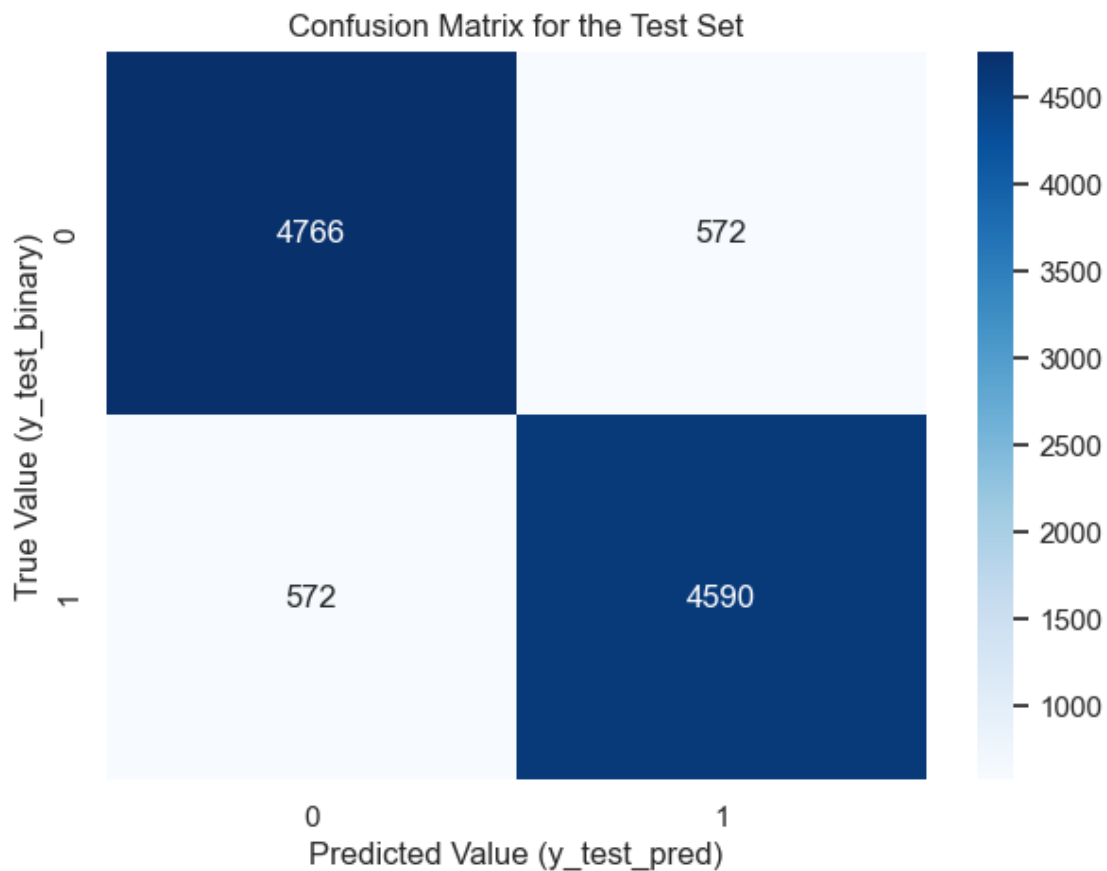
print(f'Confusion Matrix for the Test Set:\n{cm_test}')
```


Confusion Matrix for the Test Set:

```
[[4766  572]
 [ 572 4590]]
```

Visualize the confusion matrix for better readability

```
[ ]: plt.figure(figsize=(7, 5))
sns.heatmap(cm_test, annot=True, cmap='Blues', fmt='g')
plt.title('Confusion Matrix for the Test Set')
plt.xlabel('Predicted Value (y_test_pred)')
plt.ylabel('True Value (y_test_binary)')
plt.show()
```



1.8 2.7

1.8.1 Pick one random instance from those that belong to false positives and false negatives from the test set, and depict their original images in separate figures.

Find the indices of false positives and false negatives

```
[ ]: false_positives = np.where((y_test_binary == 0) & (y_test_pred == 1))[0]
      false_negatives = np.where((y_test_binary == 1) & (y_test_pred == 0))[0]
```

Select one random instance from false positives and false negatives

```
[ ]: random_fp_index = random.choice(false_positives.tolist())
      random_fn_index = random.choice(false_negatives.tolist())
```

Depict the selected false positive and false negative instances

```
[ ]: fig, axs = plt.subplots(1, 2, figsize=(8, 8))
      axs[0].imshow(X_test[random_fp_index].reshape(28, 28), cmap='gray')
      axs[0].set_title(f"False Positive Instance\nPredicted: Even / Actual: Odd")
      axs[0].axis('off')

      axs[1].imshow(X_test[random_fn_index].reshape(28, 28), cmap='gray')
      axs[1].set_title(f"False Negative Instance\nPredicted: Odd / Actual: Even")
      axs[1].axis('off')

      plt.show()
```

False Positive Instance
Predicted: Even / Actual: Odd



False Negative Instance
Predicted: Odd / Actual: Even

